

1. Diccionario inverso

Dado el diccionario:

```
extensiones = {  
    "España": "es",  
    "Francia": "fr",  
    "Alemania": "de",  
    "Italia": "it"  
}
```

Genera un segundo diccionario donde la clave sea la extensión y el valor el país.

2. Fusionar inventarios

```
inv1 = {"teclados": 12, "ratones": 20, "pantallas": 5}
```

```
inv2 = {"ratones": 4, "pantallas": 3, "altavoces": 7}
```

Fusiona los inventarios sumando cantidades cuando coincida el producto.

3. Eliminar duplicados en listas

```
contactos = {  
    "Ana": ["645123456", "645123456", "612987321"],  
    "Luis": ["622111222", "622111222", "633444555"],  
    "Marta": ["699888777", "699888777", "622333444"]  
}
```

Elimina los teléfonos repetidos de cada persona.

4. Agrupar por inicial

```
palabras = ["mesa", "móvil", "cable", "casa", "silla", "servidor", "monitor"]
```

Crea un diccionario donde las claves sean la inicial y el valor un **set** con las palabras que empiezan por esa letra.

5. Caracteres comunes

```
texto1 = "Programación en Python"
```

```
texto2 = "Aprender Python es divertido"
```

Crea un set con los caracteres comunes en ambos textos (sin espacios).

6. Filtrar diccionario por rango

```
precios = {  
    "SSD": 55,  
    "RAM": 23,  
    "CPU": 130,  
    "GPU": 240,  
    "Caja": 33,  
    "Fuente": 48  
}
```

Crea un diccionario solo con los productos cuyo precio esté entre 30 y 70.

7. Frecuencia de palabras

```
texto = "hoy es un gran día porque hoy empezamos un nuevo proyecto en Python"
```

Cuenta cuántas veces aparece cada palabra y descarta con un comprehension las que tengan menos de 3 letras.

8. Ranking de ventas

```
ventas = {  
    "Ana": 1200,  
    "Luis": 980,  
    "Marta": 1420,  
    "Jorge": 750,  
    "Lucía": 1320  
}
```

Ordena de mayor a menor usando lambda y muestra los **3 primeros**.

9. Posiciones de cada elemento

```
lista = ["rojo", "azul", "verde", "rojo", "amarillo", "azul", "rojo"]
```

Crea un diccionario donde cada color tenga una lista con las posiciones en las que aparece.

10. Dejar solo letras

`cadena = "PythOn 3.11 jes gen1al!"`

Crea una lista con solo las letras usando un list comprehension.

11. Intersección de sets

```
grupos = [  
    {"Ana", "Luis", "Marta", "Jorge"},  
    {"Marta", "Luis", "Lucía", "Pedro"},  
    {"Luis", "Marta", "Sofía"}  
]
```

Obtén un set con los nombres que aparecen en TODOS los sets.

12. Meses por año

`fechas = ["2022-03-12", "2023-01-05", "2022-03-07", "2023-02-18", "2022-09-22"]`

Crea un diccionario año → set de meses existentes.

13. Agrupar por longitud

`palabras = ["python", "java", "c", "swift", "go", "kotlin", "rust"]`

Agrupar en un diccionario por longitud usando sets.

14. Facturación total

`precios = {"barra": 1.10, "leche": 0.95, "huevos": 2.20}`

`ventas = {"barra": 30, "leche": 12, "huevos": 6}`

Calcula la facturación total (precio × unidades).

15. Vocales presentes

`texto = "Hoy estudiamos colecciones de Python"`

Obtén solo las vocales presentes (minúsculas) en un set.

16. Celsius → Fahrenheit

temperaturas = {"Madrid": 18, "Sevilla": 25, "Bilbao": 15, "Granada": 22}

Calcula un diccionario equivalente en Fahrenheit usando una dict comprehension.

17. Palabras únicas por frase

```
frases = [  
    "hoy empezamos con listas y diccionarios",  
    "las listas pueden contener muchos tipos",  
    "los diccionarios guardan pares clave valor"  
]
```

Genera una lista donde cada elemento sea un **set de palabras únicas**.

18. Ordenar tuplas

```
personas = [  
    ("Ana", 29, 167),  
    ("Luis", 22, 173),  
    ("Marta", 29, 165),  
    ("Jorge", 22, 182),  
    ("Lucía", 29, 167)  
]
```

Ordena por:
edad descendente
altura descendente
nombre ascendente

19. Filtrado doble con comprensión

numeros = [3, 8, 11, 14, 4, 2, 9, 21, 10]

Quédate solo con los pares cuyo cuadrado sea mayor que 50.

20. Aplanar matriz

```
matriz = [  
    [1, 2, 3],  
    [10, 20],
```

[7, 8, 9, 10]

]

Convierte la matriz en una sola lista usando un list comprehension doble.